ScopeFlow — Project & Requirements Specification

Version 0.1 · 13 Oct 2026 · Target: PayPal AI Hackathon submission by 12 Nov 2026 (12:00 PT); internal target 9 Nov

1. Vision

ScopeFlow is an **independent acceptance-audit platform** for project-based software work. A client and a developer agree on measurable milestone criteria; a team of specialised auditing agents gathers evidence that the delivered code and running software satisfy each criterion; both parties review the evidence; the payment state machine advances only on explicit approval or agreed resolution.

Thesis: projects should be paid against agreed, verifiable deliverables, and the verifier must be neutral — neither the author of the code nor the payer.

What ScopeFlow is not: a freelancer marketplace, a general code-review bot, or an AI judge that decides who deserves money.

2. Landscape and gap

Findings from research on 13 Oct 2026 (vendor claims are self-reported):

Category	Examples	What they answer	What they don't
Diff-level AI code review	CodeRabbit, Greptile, Qodo Merge, GitHub Copilot review, Graphite, Cursor Bugbot	"Is this change buggy, risky or messy?"	"Does the delivered work meet the contract?"
Test generation and test quality	Diffblue Testing Agent and Test Quality Agent (line, branch, mutation score, test strength); Qodo (generates tests for untested paths); open-source Stryker, PIT, mutmut	"Do tests exist, and would they catch bugs?"	Whether the tests map to agreed requirements

Category	Examples	What they answer	What they don't
Agentic E2E / QA	QA Wolf, Momentic, testRigor, QA.tech, mabl, Playwright Test Agents + MCP	"Does the app behave correctly in a browser?"	Mapping results to a two-party contract
Intent / spec verification	Aviator Verify (criteria checked by AST analysis, execution tests, LLM fallback); open-source agent-spec; SpecTrace-style traceability checkers	"Does this change match the stated intent?"	Built for one team's own agents, not two adversarial parties; no payment linkage
Freelance payment workflows	Upwork, SecureSwap, Trustless Work, FreelanceTrail (hackathon prototype: milestone escrow + GitHub verification)	Milestone funding and release	Technical evidence is thin or manual

Takeaways

1. Every single capability exists. The unoccupied position is the **combination under neutrality**: a versioned acceptance contract signed by both parties → multi-agent audit → tamper-evident evidence bundle → payment gating.
2. Aviator Verify and FreelanceTrail are the closest neighbours; ScopeFlow must differentiate on the two-party contract, the audit-agent breadth (coverage *quality*, anti-gaming, provenance) and the dispute-ready evidence bundle.
3. Industry commentary repeatedly warns that coverage numbers alone mislead and that mutation testing is the honest measure. ScopeFlow should adopt that stance.

3. The auditing agents (core of the product)

Design rules

- **Deterministic first.** Run tools and tests; use the LLM to interpret, map and explain — never to invent a pass.
- **Every finding cites an artifact** (CI run, commit SHA, file:line, log, screenshot). No citation, no finding.
- **Independence.** Auditors run in ScopeFlow's sandbox against a pinned commit, not on the developer's machine or the developer's own tests alone.

- **Agents recommend; humans approve.** No agent can move money or mark a milestone accepted.

#	Agent	Question	Method	Built on	Output
A1	**Spec Compiler**	Is the milestone testable?	LLM drafts criteria (ID, method, severity, blocking?); flags vague wording; humans edit and sign	LLM structured output	Versioned acceptance spec
A2	**Requirement Coverage Auditor**	Which code and which tests implement each criterion?	Build criterion→code→test traceability graph; LLM proposes links, tooling confirms (test names, route handlers, imports)	tree-sitter / AST, ripgrep, LLM	Trace matrix; orphan and uncovered criteria
A3	**Test Coverage Auditor**	How much of the *changed* code do tests execute?	Ingest lcov/Cobertura from CI; compute diff-coverage on lines changed in the milestone; compare with thresholds in the spec	Istanbul/c8, coverage.py, JaCoCo	Line/branch/diff coverage per file
A4	**Test Quality Auditor**	Would the tests catch real bugs?	Mutation testing on touched modules; report mutation score and surviving mutants	Stryker (JS/TS), mutmut (Python), PIT (Java)	Mutation score, surviving mutants with locations
A5	**Behaviour Auditor**	Does the deployed app actually do it?	Playwright journeys derived from criteria, run against staging URL; screenshots, traces, API checks	Playwright, Playwright MCP	Pass/fail per journey with traces
A6	**Security & Supply-Chain Auditor**	Any critical vulnerabilities, secrets, bad licences?	SAST, secret scan, dependency/OSV scan, licence check	Semgrep or CodeQL, gitleaks, Trivy/OSV-Scanner	Findings by severity
A7	**Anti-Gaming Auditor**	Is the work faked?	Detect skipped/empty/assert-nothing tests, hard-coded responses, mocked-out auth, stubbed endpoints, disabled checks in CI config, TODO/placeholder code	AST rules + LLM review	Suspicion findings with evidence

#	Agent	Question	Method	Built on	Output
A8	**Provenance Auditor**	Is what's demoed what's in the repo?	Verify deployed build hash/SHA matches audited commit; repo ownership; reproducible build; dependency lockfile present	CI metadata, GitHub API	Provenance pass/fail
A9	**Adjudication Reporter**	What is the overall status?	Aggregates evidence under the signed rules; marks each criterion *Met / Not met / Unverified / Needs human*; never outputs a single magic score	LLM + rules engine	Evidence report and recommendation

Hackathon cut: build A1, A2, A3, A5, A7 (small ruleset), A9 for real; A4 mutation testing for one language if time allows (stretch); A6 via one scanner; A8 as a SHA check.

4. Users

- **Client** — defines or accepts criteria, funds milestones, reviews evidence, approves or disputes.
- **Developer** — accepts criteria, connects repo, submits milestone, fixes failures.
- **Agency admin** (post-hackathon) — manages multiple projects and developers.
- **Reviewer/arbiter** (post-hackathon) — human reviews disputed or ambiguous criteria.

5. Functional requirements

Priority: **M** = must (hackathon), **S** = should, **C** = could (post-hackathon).

5.1 Accounts and projects

- **FR-1 (M)** Sign-in; roles Client and Developer scoped per project.
- **FR-2 (M)** Create a project with title, budget, currency, and an invite link for the counterparty.
- **FR-3 (M)** Create milestones with description, amount and due date.
- **FR-4 (C)** Organisations, multiple members, templates.

5.2 Acceptance specification

- **FR-5 (M)** Generate draft acceptance criteria from milestone text; each criterion has ID, description, verification method (automated test / agent-assisted / manual), severity, blocking flag.
- **FR-6 (M)** Flag ambiguous criteria and suggest measurable rewrites.
- **FR-7 (M)** Both parties must explicitly accept a spec version before the milestone can be funded; acceptance stored with user, timestamp and content hash.
- **FR-8 (M)** Any change creates a new version; verification always runs against a specific version.
- **FR-9 (S)** Starter templates (auth, CRUD API, payments integration, dashboard).

5.3 Repository and deployment integration

- **FR-10 (M)** GitHub App with least-privilege read access to selected repositories; webhook ingestion for push, pull request and workflow completion.
- **FR-11 (M)** Provide a GitHub Actions workflow template that runs tests and uploads coverage and test artifacts.
- **FR-12 (M)** Developer submits a milestone against a specific commit SHA and optional staging URL; audited artifacts are pinned to that SHA.
- **FR-13 (S)** Post a verification summary to the pull request as a check run.

5.4 Verification pipeline

- **FR-14 (M)** Orchestrator schedules agents A2–A9 per criterion method and collects results into an evidence record.
- **FR-15 (M)** Each evidence item stores: criterion ID, spec version, commit SHA, agent, tool and version, timestamp, result, artifact links, SHA-256 hash.
- **FR-16 (M)** Untrusted code and test execution happens only in isolated sandboxes (see §8).
- **FR-17 (M)** Re-run verification after fixes; keep full run history, nothing overwritten.
- **FR-18 (S)** Configurable thresholds per criterion (e.g. diff coverage ⩾ 80%, mutation score ⩾ 60%).

5.5 Evidence report and review

- **FR-19 (M)** Report shows each criterion with status Met / Not met / Unverified / Needs human, linked evidence and plain-language explanation.
- **FR-20 (M)** Show uncovered criteria (no evidence) distinctly from failed criteria.

- **FR-21 (M)** Client can approve, request changes (with comment tied to criteria) or open a dispute.
- **FR-22 (M)** Developer can respond, resubmit, or contest an agent finding with a comment.
- **FR-23 (S)** Downloadable evidence bundle (PDF/JSON): spec versions, approvals, runs, artifacts, decisions, timeline.

5.6 Payments

- **FR-24 (M)** Milestone state machine: Draft → Agreed → Funded → In progress → Submitted → Verifying → Review → Approved → Payment initiated → Paid; side paths Changes requested, Disputed, Cancelled.
- **FR-25 (M)** PayPal **Sandbox** order/authorisation for funding; state changes driven by verified PayPal webhooks, not client-side redirects.
- **FR-26 (M)** Release/payout call only when state is Approved; server enforces transitions.
- **FR-27 (M)** Idempotency keys on every payment call; webhook de-duplication; reconciliation job.
- **FR-28 (M)** The AI agents have no code path or credentials to initiate payment.
- **FR-29 (C)** Production payouts only after provider approval and legal review (see §9).

5.7 Disputes

- **FR-30 (S)** Dispute freezes payment actions, notifies parties, locks the current evidence set.
- **FR-31 (C)** Human arbiter workflow and resolution options (release, partial release, refund, rework).

5.8 Audit

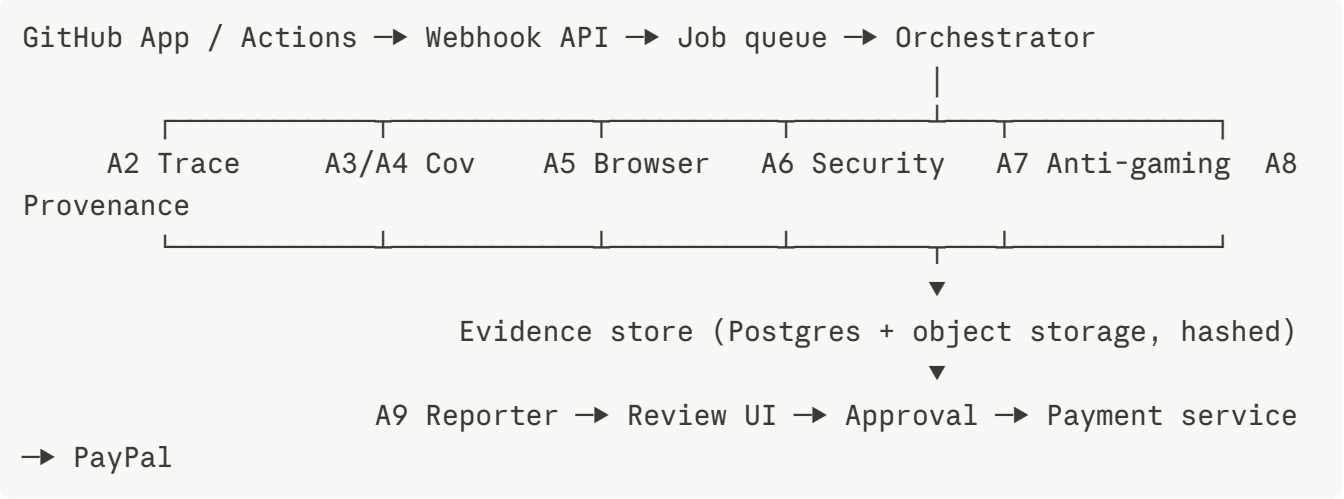
- **FR-32 (M)** Append-only audit log of every state transition, with hash chaining (each entry includes the previous entry's hash).

6. Non-functional requirements

ID	Requirement
NFR-1 Reproducibility	Same commit + same spec version + same tool versions → same deterministic results; tool and model versions recorded

ID	Requirement
NFR-2 Explainability	No finding without a linked artifact; no opaque scores
NFR-3 Security	Secrets in a vault; per-run ephemeral credentials; worker network egress allow-list; no payment secrets on workers
NFR-4 Privacy	Repo contents retained only as long as needed for the evidence window; deletion on request; clear data-use statement for LLM calls
NFR-5 Performance	Small project report in under 15 min (hackathon); live status streaming
NFR-6 Reliability	Jobs are retryable and idempotent; failed agents yield "Unverified", never a silent pass
NFR-7 Cost control	Per-run token and compute budgets; cache by commit SHA
NFR-8 Accessibility	Keyboard-navigable, readable on mobile (clients approve from phones)
NFR-9 Licensing	Public repo with an OSI-approved licence (e.g. Apache-2.0) as hackathon rules require

7. Architecture



- **Stack:** Next.js + TypeScript UI; FastAPI (or Node) API; PostgreSQL; Redis queue; S3-compatible storage; sandboxed workers (Docker with gVisor/Firecracker later); Playwright; an LLM with structured outputs and tool calling.

- **Key tables:** users, projects, milestones, spec_versions, criteria, acceptances, submissions, audit_runs, evidence_items, findings, decisions, payments, webhook_events, audit_log.
- **Payment service** is a separate module with its own credentials and no inbound path from worker processes.

8. Security and threat model

Threat	Mitigation
Malicious repo code escapes sandbox	Containerised, non-root, read-only filesystem, CPU/memory/time limits, egress allow-list
Prompt injection in code, README or comments	Treat repo text as data; agents have no payment or write tools; structured outputs validated; human approval required
Developer games the audit	A7, mutation testing, SHA/provenance pinning, ScopeFlow-owned test runs for critical criteria
Client moves goalposts	Signed, versioned specs; scope change needs both signatures
Forged webhooks / replayed payments	Signature verification, idempotency keys, de-duplication
Evidence tampering	Content hashing and hash-chained audit log
Leaked source code	Least-privilege GitHub App, short-lived tokens, encryption at rest, retention limits

9. Payments and compliance notes

- Build against **PayPal Sandbox** for the hackathon. Production multiparty/marketplace features need PayPal's partner approval.
- Holding funds pending approval is custody/escrow, which is regulated; in India it touches RBI payment-aggregator rules. Obtain legal and payments advice before handling real money. Until then, ScopeFlow can charge a subscription for verification and workflow, with payment orchestration through an approved provider.
- Describe sandbox behaviour honestly in the demo ("sandbox funds").

10. Evaluation plan (a differentiator)

Most AI tools in this space publish marketing claims. ScopeFlow should publish a small **seeded-defect benchmark**:

1. Create 5–8 small reference repos with a milestone spec and a known-correct implementation.
2. Inject defects: missing feature, wrong behaviour, vulnerable dependency, assert-nothing tests, hard-coded responses, skipped tests, deployed build ≠ repo commit.
3. Run the audit; measure **recall** (defects caught), **precision** (false alarms), and **unverified rate**.
4. Report results per agent in the README and demo.

11. Hackathon plan (13 Oct → 12 Nov)

Week	Goal
1 (13–19 Oct)	Repo + licence; auth; project/milestone/spec CRUD; spec compiler (A1); versioned dual acceptance; PayPal sandbox account and funding flow
2 (20–26 Oct)	GitHub App + Actions template; submission pinned to SHA; sandbox worker; A3 diff-coverage; A5 Playwright journey for the demo app
3 (27 Oct–2 Nov)	A2 trace matrix; A7 anti-gaming rules; A9 report UI; approve/changes flow; payment state machine + webhooks + idempotency
4 (3–9 Nov)	Seeded-defect benchmark; evidence-bundle export; polish; demo repo; README, architecture diagram; record 3-minute video; submit with buffer

Cut order if late: mutation testing → dispute workflow → PR check-run → evidence PDF (keep JSON).

12. Demo script (under 3 minutes)

1. Client and developer accept a 5-criterion authentication milestone (₹20,000 sandbox funding via PayPal Sandbox).
2. Developer pushes code with a hidden flaw: password reset untested and one test asserting nothing.

3. ScopeFlow audit runs: diff coverage low, A7 flags the empty test, A5 shows reset flow failing; report marks criteria Not met / Unverified with evidence.
4. Developer fixes, resubmits; report turns green; evidence is hash-linked.
5. Client approves; state moves to Payment initiated → Paid via sandbox webhook.
6. Show the evidence bundle and the seeded-benchmark results slide.

13. Success metrics

- Demo: all five stages work live against real GitHub and PayPal Sandbox.
- Benchmark: recall and precision reported honestly per agent.
- Post-hackathon pilots: time from submission to acceptance, rework cycles per milestone, disputes per 100 milestones, share of findings clients rate as useful.

14. Open questions

1. Which language/framework does the demo app use (recommend TypeScript + Playwright for simplest end-to-end story)?
2. Which LLM, and how are keys and costs capped during judging?
3. How should "manual" criteria (design quality, UX) be handled — explicit client sign-off only?
4. Validate with 5–10 agencies or independent developers before building beyond the hackathon scope: are acceptance disputes frequent and costly, and would they trust automated evidence?